

Universidad de la Cuenca del Plata

Facultad de Ingeniería y Tecnología · Ingeniería en Sistemas de Información

Simulador de procesos y memoria con POO

Actividad de Evaluación 2 · Paradigmas y Lenguajes de Programación II

Dato	Detalle
Estudiante	Mariano Ifran
DNI	46923088
Carrera	Ingeniería en Sistemas de Información
Asignatura	Paradigmas y Lenguajes de Programación II
Docentes	José A. Fernández · S. Romina Stacciuoli
Fecha de entrega	05/10/2026
Repositorio	https://github.com/MarianIFran/planificador-cpu-memoria
Versión evaluada	Tag v1.0.0

1. Objetivo y escenario

El objetivo del trabajo es aplicar programación orientada a objetos para convertir los requerimientos de un simulador de Sistemas Operativos en un modelo de clases cohesivo, encapsulado y verificable con pruebas automatizadas.

El escenario es un laboratorio universitario que necesita un motor para estudiar cómo varios procesos comparten una memoria limitada y una única CPU. Los procesos piden memoria, esperan su turno, consumen CPU, pueden bloquearse por Entrada/Salida y liberan sus recursos al terminar.

La solución es una biblioteca de clases escrita en TypeScript. No tiene interfaz gráfica, menú de consola ni función main: su funcionamiento se demuestra únicamente con los tests. La simulación empieza en el tick 0, con la memoria vacía y la CPU libre, y avanza por ticks discretos, sin reloj real, hilos ni azar. La configuración de referencia es 1024 KB de memoria y un quantum de 2 ticks; ambos valores se pueden cambiar.

2. Diseño y responsabilidades

El código de producción está en la carpeta src y los tests en la carpeta tests, con la misma organización. El comportamiento se reparte entre varios objetos, de manera que ninguna clase concentre toda la lógica.

Clase o interfaz	Responsabilidad
Simulador	Coordina la simulación. Registra procesos y ordena las cuatro fases de cada tick. No asigna memoria ni planifica: se lo pide al objeto que corresponde.
Proceso	Guarda los datos de un proceso y protege sus cambios de estado y sus contadores.
EstadoProceso	Enumeración con los seis estados posibles.
EventoES	Evento de Entrada/Salida: tras cuántos ticks de CPU se dispara y cuánto dura.
AdministradorMemoria (interfaz)	Contrato de la gestión de memoria que usa el simulador.
GestorMemoria	Mantiene la lista ordenada de bloques. Asigna, libera y fusiona huecos vecinos.
BloqueMemoria	Bloque contiguo e inmutable: inicio, tamaño y proceso que lo ocupa (o libre).
PoliticaAsignacion (interfaz)	Contrato común para elegir en qué hueco ubicar un pedido.
FirstFit, BestFit, WorstFit	Las tres políticas de asignación contigua.
PlanificadorCpu (interfaz)	Contrato de la planificación de CPU que usa el simulador.
PlanificadorRoundRobin	Administra la CPU y la cola FIFO de Listos, y cuenta los cambios de contexto.
CalculadorMetricas (interfaz) y CalculadorMetricasEstandar	Aplican las fórmulas de las métricas.
ProcesoLectura, BloqueLectura, MetricasLectura (interfaces)	Vistas de solo lectura que se entregan hacia afuera.
ErrorSimulacion	Error que se lanza cuando se rompe una regla del dominio.

2.1. Diagrama de clases

El diagrama completo, con atributos, métodos, visibilidad, interfaces, relaciones y multiplicidades, está en el repositorio en dos versiones: imagen (docs/diagramas/clases.png) y fuente editable en formato Mermaid (docs/diagramas/clases.mmd).

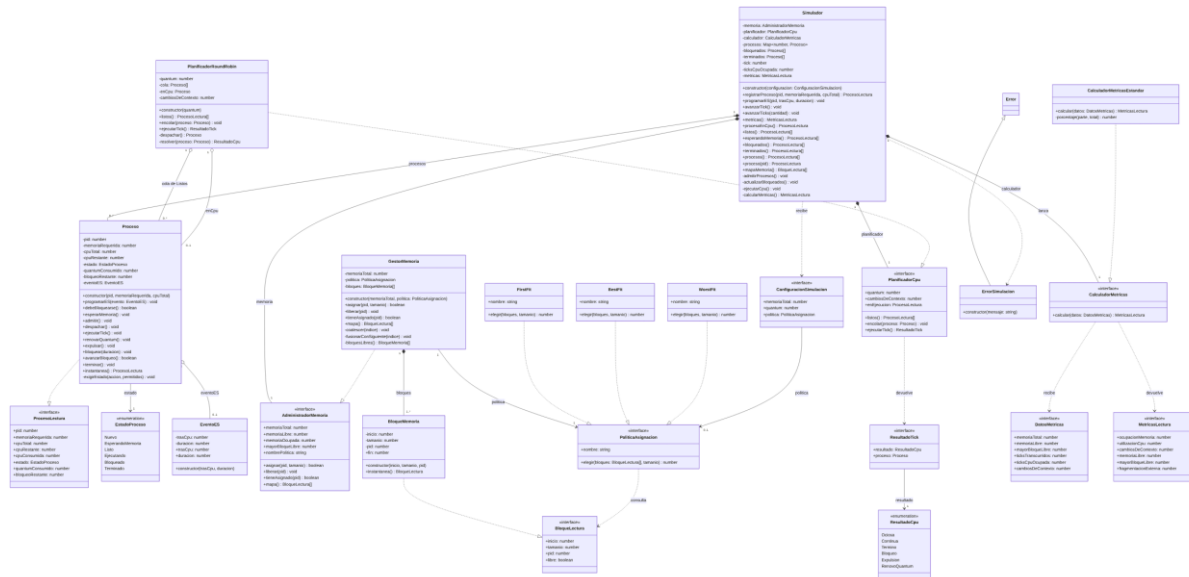


Figura 1. Diagrama de clases (versión ampliable en docs/diagramas/clases.png).

3. Decisiones de programación orientada a objetos

3.1. Encapsulamiento

Todos los atributos son privados (se declaran con #). El estado solo cambia mediante métodos que validan las reglas del dominio. Por ejemplo, `Proceso.despachar()` solo funciona si el proceso está en `Listo`, y `Proceso.terminar()` solo si ya consumió toda su CPU.

Además, las consultas nunca entregan los objetos internos. `Proceso.instantanea()`, `GestorMemoria.mapa()` y las consultas del Simulador devuelven copias congeladas con `Object.freeze`. Los tests verifican que intentar modificarlas lanza un error y que el estado real no cambia.

3.2. Interfaces y principios SOLID

- **Responsabilidad única:** cada clase tiene un solo motivo para cambiar (memoria, CPU, métricas, coordinación).
- **Abierto/cerrado:** se puede agregar una política de asignación nueva sin modificar `GestorMemoria`.
- **Sustitución de Liskov:** cualquier objeto que cumpla `PoliticaAsignacion` puede reemplazar a otro. En los tests se usa una política falsa que funciona igual que las reales.
- **Segregación de interfaces:** las interfaces son chicas y específicas. Las de lectura (`ProcesoLectura`, `BloqueLectura`) no incluyen métodos que modifiquen.
- **Inversión de dependencias:** el Simulador declara sus atributos con los tipos `AdministradorMemoria`, `PlanificadorCpu` y `CalculadorMetricas`, no con las clases concretas.

3.3. Polimorfismo

`GestorMemoria.asignar()` llama a `politica.elegir(bloques, tamaño)` sin saber qué política recibió y sin ningún condicional por tipo. El mismo mensaje produce tres comportamientos distintos. En los tests se ve de dos maneras: el mismo pedido de 80 KB queda en direcciones distintas según la política, y una política de prueba creada dentro del test es aceptada por el gestor.

3.4. Composición y colecciones

El Simulador tiene una memoria, un planificador y un calculador de métricas, y guarda sus procesos en un Map ordenado por registro. El GestorMemoria tiene una lista de BloqueMemoria. El PlanificadorRoundRobin tiene una cola de procesos Listos. Ninguna de estas colecciones se expone: hacia afuera solo salen copias de solo lectura.

3.5. Herencia y clases abstractas

La única herencia es ErrorSimulacion extends Error, porque un error del simulador sí es un Error y puede usarse donde se espera uno. No se armó una jerarquía de procesos ni de estados: un proceso cambia de estado pero no de tipo, así que el estado se modela con una enumeración.

Tampoco se usaron clases abstractas. Las tres políticas no comparten código, solo el contrato, por lo que una interfaz es suficiente. Una clase abstracta habría agregado una base común sin nada que compartir.

4. Política de memoria

La memoria se representa como una lista de bloques ordenada por dirección. Cada bloque tiene inicio, tamaño y el PID del proceso que lo ocupa, o está libre. La lista cumple siempre tres condiciones: los bloques son contiguos, no se solapan y la suma de sus tamaños es igual a la memoria total.

Se implementaron las tres políticas, que se eligen al configurar la simulación:

- **First-Fit:** el primer bloque libre suficiente, recorriendo por dirección.
- **Best-Fit:** el bloque libre suficiente de menor tamaño.
- **Worst-Fit:** el bloque libre suficiente de mayor tamaño.

Ante un empate, las tres eligen el bloque de menor dirección. Si el bloque elegido es más grande que el pedido, se divide en una parte ocupada y un sobrante libre. Un ajuste exacto no genera bloques de tamaño cero. Si ningún hueco contiguo alcanza, la asignación falla sin modificar los bloques, aunque la suma de memoria libre sea suficiente.

4.1. Liberación y coalescencia

Cuando un proceso termina, su bloque se marca como libre y se fusiona con los vecinos libres. La coalescencia no mueve bloques ocupados, por lo que no es compactación. El mecanismo es el siguiente:

```
liberar(pid):
  i <- posicion del bloque ocupado por pid
  si no existe: error
  bloques[i] <- bloque libre con el mismo inicio y tamaño
  fusionarConSiguiente(i)    // une con el vecino derecho
  fusionarConSiguiente(i - 1) // une con el vecino izquierdo

fusionarConSiguiente(i):
  si bloques[i] y bloques[i + 1] existen y los dos estan libres:
    reemplazar ambos por un solo bloque libre
    con inicio = bloques[i].inicio
    y tamaño = bloques[i].tamaño + bloques[i + 1].tamaño
```

Se fusiona primero a la derecha y después a la izquierda para que el índice del bloque liberado siga siendo válido en el segundo paso.

5. Estados y orden por tick

Estado	Significado
Nuevo	Proceso recién registrado; todavía no se intentó asignarle memoria.

Estado	Significado
Esperando Memoria	No hay un bloque contiguo suficiente. Se reintenta al inicio de cada tick.
Listo	Tiene memoria asignada y espera en la cola para usar la CPU.
Ejecutando	Está usando la CPU. Como máximo hay uno.
Bloqueado	Está en una Entrada/Salida. Conserva su memoria y no consume CPU.
Terminado	Consumió toda su CPU y liberó su memoria. No vuelve a las colas.

Transiciones permitidas: Nuevo → Listo; Nuevo → Esperando Memoria → Listo; Listo → Ejecutando; Ejecutando → Listo (quantum agotado); Ejecutando → Bloqueado → Listo (Entrada/Salida); Ejecutando → Terminado. Cualquier otra transición lanza un ErrorSimulacion.

Cada llamada a Simulador.avanzarTick() avanza exactamente una unidad de tiempo y ejecuta siempre las mismas fases:

1. **Admisión:** intenta asignar memoria a los procesos Nuevos y en espera, en orden de registro. Un proceso que no cabe no impide admitir a otro que sí cabe.
2. **Bloqueados:** descuenta los temporizadores de E/S. Los que llegan a cero vuelven al final de la cola de Listos y pueden ser despachados en ese mismo tick.
3. **CPU:** el planificador despacha si la CPU está libre y ejecuta una unidad de CPU. Después decide con esta prioridad: finalización, bloqueo por E/S, agotamiento del quantum.
4. **Reloj y métricas:** suma un tick y recalcula las métricas.

Por este orden, la memoria que libera un proceso al terminar recién se ofrece a los que esperan en la fase de admisión del tick siguiente.

5.1. Round-Robin y cambios de contexto

Al despachar un proceso se reinicia su quantum consumido. Si agota el quantum y hay otros procesos Listos, vuelve al final de la cola. Si no hay otros Listos, renueva su quantum y sigue en la CPU. Se cuenta un cambio de contexto por cada expulsión por quantum con otros Listos y por cada bloqueo de E/S. No se cuentan el despacho inicial, la finalización ni la renovación de quantum.

5.2. Validación de los eventos de Entrada/Salida

Un evento indica tras cuántos ticks de CPU consumidos se dispara y cuánto dura. Se rechaza si:

- el disparo o la duración no son enteros positivos;
- el proceso ya tiene un evento (cada proceso admite uno solo);
- el proceso está Terminado;
- el disparo es mayor que la CPU total del proceso (nunca ocurriría);
- el disparo ya quedó en el pasado respecto de la CPU consumida.

Si el disparo coincide con el último tick de CPU, el proceso termina y no se bloquea, porque la finalización tiene prioridad.

6. Diagramas de secuencia

Se entregan tres diagramas, cada uno asociado a requerimientos distintos. Los objetos y mensajes son los de la implementación. Las fuentes editables están en docs/diagramas con extensión .mmd.

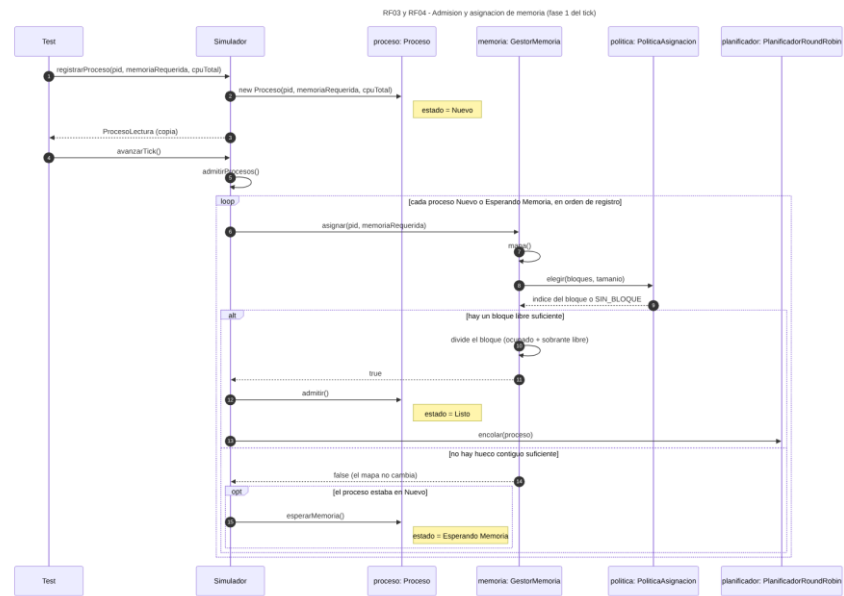


Figura 2. RF03 y RF04: admisión y asignación de memoria.

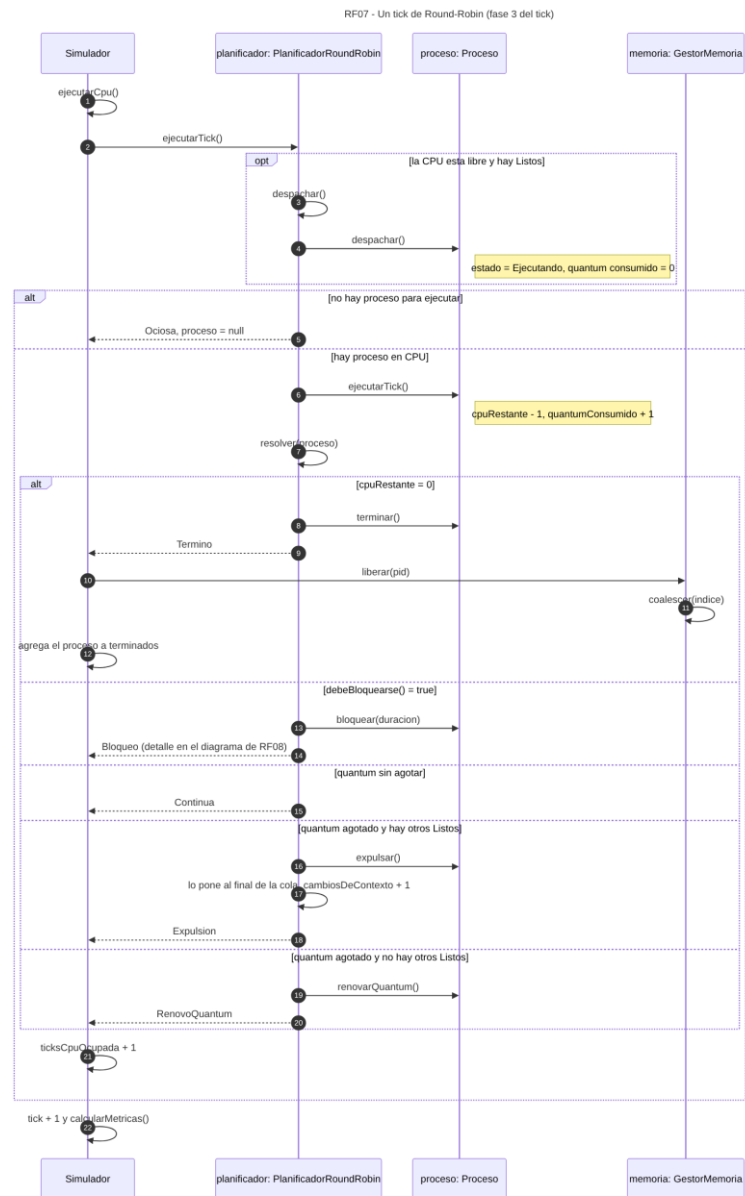


Figura 3. RF07: un tick de Round-Robin.

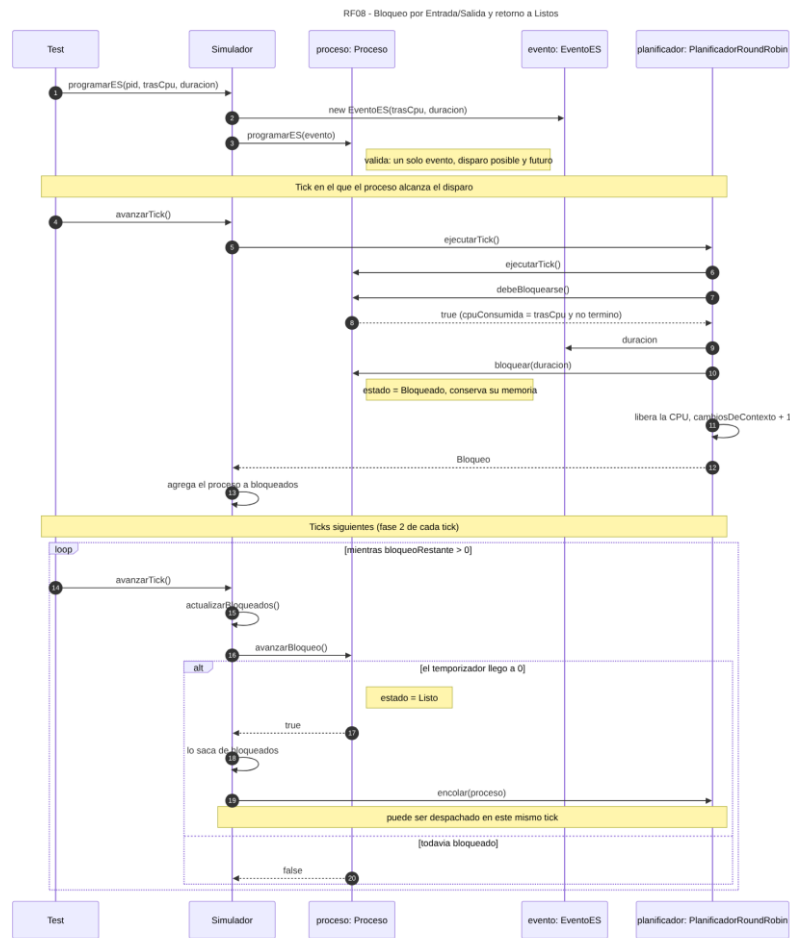


Figura 4. RF08: bloqueo por Entrada/Salida y retorno a Listos.

7. Matriz de requerimientos, clases y tests

RF	Clase o método	Tests
RF01	Simulador (constructor), ConfiguracionSimulacion	Simulador.test.ts (configuración e inicio), GestorMemoria.test.ts (estado inicial), PlanificadorRoundRobin.test.ts (configuración)
RF02	Simulador.registrarProceso, Simulador.proceso, Proceso, ProcesoLectura	Simulador.test.ts (registro), Proceso.test.ts (creación, consulta protegida)
RF03	EstadoProceso, transiciones de Proceso, Simulador.#admitirProcesos	Proceso.test.ts (transiciones), Simulador.test.ts (estados y admisión)
RF04	GestorMemoria.asignar, PoliticaAsignacion, FirstFit, BestFit, WorstFit, BloqueMemoria	Políticas.test.ts, GestorMemoria.test.ts, BloqueMemoria.test.ts, Coalescencia.test.ts (selección según política), ComparacionPolíticas.test.ts
RF05	GestorMemoria.liberar, #coalescer, #fusionarConSiguiente	Coalescencia.test.ts
RF06	Simulador.avanzarTick, Simulador.avanzarTicks	Simulador.test.ts (avance por ticks)
RF07	PlanificadorCpu, PlanificadorRoundRobin	PlanificadorRoundRobin.test.ts, Simulador.test.ts (avance por ticks)
RF08	EventoES, Proceso.programarES, Proceso.debeBloquearse,	EventoES.test.ts, PlanificadorRoundRobin.test.ts (bloqueo), Simulador.test.ts (Entrada/Salida)

RF	Clase o método	Tests
	Simulador.programarES, Simulador.#actualizarBloqueados	
RF09	CalculadorMetricas, CalculadorMetricasEstandar, MetricasLectura, Simulador.metricas	CalculadorMetricas.test.ts, Simulador.test.ts (métricas), ComparacionPolíticas.test.ts
RF10	Consultas del Simulador: tickActual, procesoEnCpu, listos, esperandoMemoria, bloqueados, terminados, mapaMemoria	Simulador.test.ts, GestorMemoria.test.ts (mapa protegido)

8. Resultados y cobertura

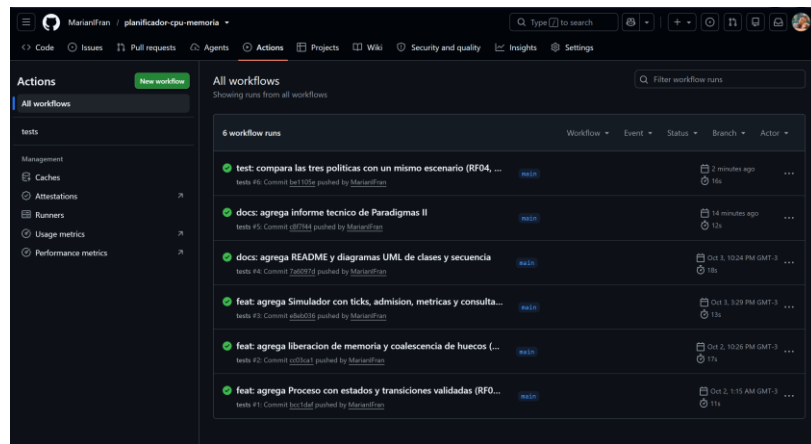
Dato	Detalle
Herramienta	Vitest, con el proveedor de cobertura @vitest/coverage-v8
Comando	npm run coverage
Alcance medido	Todos los archivos de src/**/*.ts, incluso los que ningún test importe (configurado en vitest.config.ts)
Umbral	El comando falla si la cobertura de líneas es menor a 91%
Tests	203 tests en 10 archivos, todos aprobados
Cobertura de líneas	100 %
Integración continua	GitHub Actions corre el chequeo de tipos y la cobertura en cada push (.github/workflows/tests.yml)

Los archivos que solo contienen interfaces figuran con 0 en el reporte porque no tienen líneas ejecutables: TypeScript las elimina al compilar y no afectan el total.

```
>> New item - Item type: Directory - Force (Split-Path $?) | Out-Null
>> [IO.File]: WriteAllText($?, $texto) ...
Test Files: 10 passed (10)
Tests: 203 passed (203)
Start at 19:38:52
Duration 1.02s (transform 53%, import 22%, worker 18%, tests 7%)

% Coverage report from v8
```

File	% Stats	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
src	100	100	100	100	
errores.ts	100	100	100	100	
index.ts	0	0	0	0	
src/memoria	100	100	100	100	
AdministradorMemoria.ts	0	0	0	0	
BestFit.ts	100	100	100	100	
BloqueLectura.ts	0	0	0	0	
BloqueMemoria.ts	100	100	100	100	
FirstFit.ts	100	100	100	100	
GestorMemoria.ts	100	100	100	100	
PoliticaAsignacion.ts	100	100	100	100	
WorstFit.ts	100	100	100	100	
src/metricas	100	100	100	100	
CalculadorMetricas.ts	100	100	100	100	
MetricasLectura.ts	0	0	0	0	
src/planificacion	100	100	100	100	
PlanificadorCpu.ts	100	100	100	100	
PlanificadorRoundRobin.ts	100	100	100	100	
src/proceso	100	100	100	100	
EstadoProceso.ts	100	100	100	100	
EventoE5.ts	100	100	100	100	
Proceso.ts	100	100	100	100	
ProcesoLectura.ts	0	0	0	0	
src/simulador	100	100	100	100	
ConfiguracionSimulacion.ts	100	100	100	100	
Simulador.ts	100	100	100	100	



8.1. Casos mínimos de verificación

Caso de la consigna	Dónde se comprueba
Configuración y registro	Simulador.test.ts: estado inicial, parámetros inválidos, PID repetido, proceso mayor que la memoria.
Asignación y espera	GestorMemoria.test.ts: división, ajuste exacto, fallo sin modificar. Simulador.test.ts: admisión posterior al liberarse memoria.
Coalescencia	Coalescencia.test.ts: fusión izquierda, derecha y ambas; al liberar todo queda un único bloque del tamaño total.
Round-Robin	Q = 2, P1 con CPU 3 y P2 con CPU 2: ejecución P1, P1, P2, P2, P1 y un cambio de contexto.
Quantum y finalización	Un único proceso renueva quantum sin cambio de contexto; terminar en el límite de quantum no reencola.
Bloqueo por E/S	Se bloquea, conserva la memoria, no consume CPU y vuelve a Listos; se verifica el contador de contexto.
Métricas y límites	Huecos de 100 y 300 KB: libre 400, mayor 300, fragmentación 25%. También memoria llena y tick 0.
Orden e invariantes	Una liberación al final del tick habilita la admisión en el siguiente. Sin duplicados, solapamientos ni dos procesos en CPU.

9. Análisis de fragmentación y comparación de políticas

La fragmentación externa aparece cuando la memoria libre está repartida en huecos que no son vecinos. Se calcula como $100 \times (1 - \text{mayor bloque libre} / \text{memoria libre total})$; si no hay memoria libre vale 0.

El caso de referencia de la consigna se reproduce en los tests: con huecos no contiguos de 100 KB y 300 KB hay 400 KB libres, pero el mayor hueco es de 300 KB, lo que da 25% de fragmentación. Un pedido de 400 KB falla aunque la suma alcance. La coalescencia reduce la fragmentación solo cuando los huecos son vecinos; si entre ellos hay un bloque ocupado, no puede unirlos porque no mueve procesos.

9.1. Mismo escenario con las tres políticas

El test ComparacionPoliticas.test.ts corre el mismo escenario con cada política: memoria de 1000 KB y quantum 1. Después de tres ticks quedan dos huecos, uno de 300 KB en la dirección 0 y otro de 100 KB en la dirección 500. Luego llega P5, que pide 80 KB, y un tick más tarde P6, que pide 250 KB.

Política	Dónde ubica a P5	Mayor hueco después	Fragmentación	¿P6 (250 KB) es admitido?
First-Fit	Dirección 0	220 KB	31,25 %	No, queda Esperando Memoria
Best-Fit	Dirección 500	300 KB	6,25 %	Sí
Worst-Fit	Dirección 0	220 KB	31,25 %	No, queda Esperando Memoria

First-Fit y Worst-Fit usan el hueco de 300 KB para un pedido chico y lo dejan en 220 KB, por lo que P6 ya no entra aunque haya 320 KB libres. Best-Fit ubica a P5 en el hueco de 100 KB y conserva entero el de 300 KB, así que P6 es admitido.

En este escenario Best-Fit resulta mejor, pero no es una regla general. Best-Fit tiende a dejar sobrantes muy chicos (aquí, uno de 20 KB) que son difíciles de reutilizar. First-Fit es el más simple y rápido porque corta en el primer hueco que sirve. Worst-Fit deja sobrantes grandes, pero consume primero los huecos grandes y perjudica a los pedidos grandes que llegan después. El resultado depende del orden y del tamaño de los pedidos.

10. Conclusiones

La biblioteca cumple los diez requerimientos funcionales y todos están verificados con tests independientes, con casos normales, límites y de error. La cobertura de líneas supera el mínimo pedido y la integración continua corre los tests en cada push.

Separar las responsabilidades en objetos chicos hizo que cada parte se pudiera probar sola: las políticas con mapas armados a mano, el gestor de memoria sin planificador, y el planificador sin memoria. Dependier de interfaces permitió, por ejemplo, probar el gestor con una política falsa.

El encapsulamiento resultó ser la decisión más importante. Como el estado de un proceso solo cambia con métodos que validan, los errores de orden aparecen como excepciones claras y no como estados inconsistentes difíciles de rastrear.

11. Fuentes consultadas

- Silberschatz, A., Galvin, P. B. y Gagne, G. Fundamentos de sistemas operativos. McGraw-Hill. Capítulos de planificación de CPU y gestión de memoria principal.
- Tanenbaum, A. S. y Bos, H. Sistemas operativos modernos. Pearson. Capítulos de procesos y administración de memoria.
- Stallings, W. Sistemas operativos: aspectos internos y principios de diseño. Pearson.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. Patrones de diseño. Addison-Wesley. Patrón Strategy, usado para las políticas de asignación.
- Documentación de TypeScript: <https://www.typescriptlang.org/docs/>
- Documentación de Vitest (tests y cobertura): <https://vitest.dev/>
- Documentación de GitHub Actions: <https://docs.github.com/actions>
- Documentación de Mermaid (diagramas): <https://mermaid.js.org/>
- Material de clase y consignas del AE2 de Paradigmas y Lenguajes de Programación II y de Sistemas Operativos (UCP, 2026).